

RAVEN — Avaliação Didática

ddnr03@gmail.com [Mudar de conta](#)



✉ Não compartilhado

* Indica uma pergunta obrigatória

Seção 4 — Pós-Teste

Responda com base no que foi trabalhado durante a sessão com o RAVEN. Não consulte materiais externos. As questões são as mesmas da seção inicial — responda de forma independente. Tempo estimado: 15 minutos.

Questão 1 — Formatos de instrução RISC-V (arquitetura de conjunto de instruções aberta e reduzida) *

Quais formatos de instrução RISC-V (arquitetura de conjunto de instruções aberta e reduzida) não possuem o campo rd (registrador de destino) em sua codificação binária?

- ☐ A) R-type (add, sub, mul) e I-type (addi, lw, jalr) — porque usam dois operandos fonte e poderiam reaproveitar o espaço de rd para ampliar o imediato.
- ☐ B) S-type (sw, sb) e B-type (beq, bne) — porque produzem efeito em memória ou no PC (Contador de Programa) e não escrevem em registrador de destino. Nesses formatos, os bits [11:7] são reaproveitados para codificar parte do imediato.
- ☐ C) U-type (upper immediate) e J-type (jump) — porque o imediato de 20 bits ocupa quase toda a instrução e empurra o campo rd para fora da codificação útil.
- ☐ D) Apenas ECALL (chamada ao ambiente) e EBREAK (ponto de parada) — porque instruções de sistema são tratadas por um caminho especial de controle e, por isso, dispensam qualquer registrador de destino.

Questão 2 — Função do campo tag no endereçamento de cache *

Um endereço de memória é dividido em: offset, índice e tag. Qual desses campos verifica se o bloco armazenado em um set corresponde ao endereço sendo acessado?

- ☐ A) O campo índice, porque identifica unicamente o set e — em mapeamento direto — também identifica unicamente o bloco armazenado naquele set.
- ☐ B) O campo offset, porque indica a posição exata dentro do bloco.
- ☐ C) O campo tag, porque múltiplos endereços distintos mapeiam para o mesmo set. A tag carrega os bits de alta ordem e permite confirmar se o bloco presente naquele set pertence ao endereço sendo acessado.
- ☐ D) Nenhum campo adicional é necessário em mapeamento direto — o índice já garante que o bloco é o correto.

Questão 3 — Quando LRU deixa de ajudar *

Por que a política LRU falha em programas que percorrem arrays grandes em sequência?

LRU (Menos Recentemente Usado). FIFO (Primeiro a Entrar, Primeiro a Sair).

- ☐ A) Porque LRU exige mais hardware que FIFO, e esse custo extra pode anular qualquer ganho quando o array é muito grande.
- ☐ B) Porque, nesse padrão de acesso, o programa quase não reutiliza os blocos logo depois de acessá-los. Sem reutilização de curto prazo, guardar o "mais recente" não traz vantagem real.
- ☐ C) Porque isso só acontece quando o array não cabe na cache; se couber, LRU vira automaticamente a melhor escolha para qualquer acesso sequencial.
- ☐ D) Porque acessos sequenciais sempre ativam um prefetch automático que torna a política de substituição praticamente irrelevante.



Questão 4 — Como o AMAT (Tempo Médio de Acesso à Memória) pode piorar mesmo com menos misses *

Considere duas caches L1 (Nível 1).
A Config A tem hit time de 1 ciclo e miss rate de 8%.
A Config B tem hit time de 4 ciclos e miss rate de 6%.
As duas têm miss penalty de 50 ciclos.

Se a Config B erra menos acessos, como o AMAT dela ainda pode ficar pior?

- ☐ A) Porque o AMAT combina o custo dos hits com o custo dos misses. Se cada hit ficar muito mais caro, essa piora pode superar a pequena redução no miss rate.
- ☐ B) Porque isso só pode acontecer se houver erro de configuração; em L1, hit time acima de 1 ciclo já invalidaria a comparação.
- ☐ C) Porque, quando há muitos acessos à memória, o miss penalty domina tanto o cálculo que o hit time quase deixa de importar.
- ☐ D) Porque caches maiores sempre deveriam ter AMAT menor; se isso não aconteceu, a fórmula do AMAT não serve para comparar as duas.

Questão 5 — Função do estágio WB (Escrita de Volta) no pipeline de cinco estágios *

No pipeline IF (Busca da Instrução) → ID (Decodificação da Instrução) → EX (Execução) → MEM (Acesso à Memória) → WB (Escrita de Volta). Qual é a função primária do estágio WB (Escrita de Volta)?

- ☐ A) Ler os operandos do banco de registradores (rs1, rs2) e detectar dependências de dados entre instruções em voo.
- ☐ B) Escrever o resultado da operação no registrador de destino (rd) especificado pela instrução.
- ☐ C) Buscar a próxima instrução na memória de instruções usando o valor atual do PC (Contador de Programa).
- ☐ D) Executar a operação aritmética ou lógica na ULA sobre os operandos preparados no estágio anterior.

Questão 6 — O que o pipeline realmente melhora *

Por que a frase "o pipeline acelera o programa porque cada instrução individual fica mais rápida" está errada?

- ☐ A) Está incorreta porque o pipeline reduz a frequência de clock para acomodar os múltiplos estágios, e o ganho vem apenas do paralelismo entre programas diferentes.
- ☐ B) Está incorreta porque o ganho vem exclusivamente do cache de instruções (I-Cache (Cache de Instruções)) — o pipeline em si não contribui.
- ☐ C) Está incorreta porque o pipeline aumenta o throughput (instruções finalizadas por unidade de tempo) ao sobrepor execução de múltiplas instruções em estágios distintos, mas a latência individual de cada instrução permanece a mesma ou até aumenta levemente.
- ☐ D) Está incorreta porque o pipeline só acelera instruções aritméticas; instruções de memória e branches têm a mesma latência em execução pipelined e sequencial.

Questão 7 — Por que ainda há bolha no caso load-use *

lw x5, 0(x1)
add x6, x5, x2

Com pipeline e forwarding ativos, por que uma bolha ainda aparece?
Nesta questão, WAW (Escrita Após Escrita).

- ☐ A) O forwarding não foi ativado corretamente — a bolha sugere que o valor de x5 deixou de ser encaminhado a tempo do estágio que faria a soma, algo resolvível pela configuração.
- ☐ B) Formam um hazard WAW (Escrita Após Escrita), pois as duas instruções passam pelo registrador x5 e o pipeline precisa serializar esse conflito de escrita.
- ☐ C) O branch predictor detectou um possível desvio e inseriu a bolha preventivamente antes de confirmar que não havia nenhum branch relevante no trecho.
- ☐ D) O lw só entrega o valor de x5 ao final de MEM (Acesso à Memória), mas o add precisa dele no início de EX (Execução). Mesmo com forwarding (encaminhamento), o dado ainda não existe no ciclo necessário, então exatamente 1 stall continua inevitável no caso load-use.



Questão 8 — Stall e flush podem parecer parecidos, mas não são a mesma coisa *

Por que (1) uma instrução parada com bolhas à frente e (2) várias instruções descartadas no meio do caminho têm causas diferentes?

- ☐ A) Os dois eventos são a mesma coisa — nomes diferentes para qualquer ciclo improdutivo, sem distinção real entre causa, momento de detecção ou mecanismo interno.
- ☐ B) O primeiro ocorre apenas sem forwarding; o segundo só com forwarding ativado, já que os dois comportamentos são mutuamente exclusivos na lógica de controle do pipeline.
- ☐ C) O primeiro é um stall por hazard de dados: o pipeline segura uma instrução e injeta bolhas para esperar o operando. O segundo é um flush por hazard de controle: instruções já buscadas são descartadas porque vieram de um caminho errado, como após branch mal previsto ou exceção.
- ☐ D) O primeiro ocorre quando a D-Cache (Cache de Dados) tem um miss; o segundo, quando a I-Cache (Cache de Instruções) tem um miss, então ambos são variações de latência do subsistema de memória.

Questão 9 — Que dependência aparece nesse trecho? *

mul x5, x1, x2
add x6, x5, x3

Que tipo de hazard de dados aparece aqui, e por quê?

- ☐ A) WAW (Escrita Após Escrita): como ambas escrevem resultados em registradores, o pipeline detecta um conflito de escrita e força a serialização da confirmação final.
- ☐ B) WAR (Escrita Após Leitura): o add lê x5 enquanto o mul ainda está produzindo esse valor, criando o risco de uma escrita posterior atrapalhar a leitura já iniciada.
- ☐ C) Nenhum hazard relevante: como os destinos finais são diferentes, a dependência observada é apenas aparente e não exige tratamento especial do hardware.
- ☐ D) RAW (Leitura Após Escrita): o add precisa ler x5 como fonte antes de o mul terminar de escrevê-lo como destino. Em pipeline em ordem, essa dependência de dado é a que gera stall real e precisa de forwarding (encaminhamento) ou espera.

Questão 10 — O que o CPI (Ciclos Por Instrução) mede e o que ele não mede *

O painel exibe "CPI: 1,85". Por que a interpretação "cada instrução demorou 1,85 segundos" está errada?
CPI (Ciclos Por Instrução).

- ☐ A) CPI é medido em nanosegundos, não em segundos; logo 1,85 já descreveria diretamente o tempo médio por instrução em hardware moderno.
- ☐ B) CPI é a média de ciclos de clock gastos por instrução ao longo da execução. Um CPI de 1,85 significa 1,85 ciclos por instrução em média; para virar tempo, ainda falta considerar o número de instruções e a frequência do clock.
- ☐ C) A interpretação está parcialmente correta: como ciclos modernos costumam ser muito curtos, o valor numérico do CPI pode ser lido aproximadamente como tempo em nanossegundos por instrução.
- ☐ D) CPI de 1,85 significa que o pipeline executou 1,85 instruções por ciclo, isto é, um IPC (Instruções Por Ciclo) elevado apesar das bolhas e dos stalls observados.

Questão 11 — Por que o speedup real fica abaixo do ideal *

Um pipeline de 5 estágios produz speedup real de 2,8x, e não 5x. O que explica essa diferença?

- ☐ A) O speedup é limitado principalmente pelo barramento de memória: sem pipeline, a memória efetiva fica relativamente mais rápida e compensa boa parte da vantagem teórica do paralelismo entre estágios.
- ☐ B) O speedup real seria 5x se o programa não tivesse branches; como há desvios, eles se tornam o único fator relevante para derrubar o ganho abaixo do valor ideal.
- ☐ C) O speedup de 5x pressupõe que apenas instruções independentes sejam sobrepostas; sempre que surge qualquer dependência, o hardware volta ao comportamento quase sequencial e perde paralelismo.
- ☐ D) O speedup ideal de Nx assume um pipeline perfeito e sem overhead. Na prática, stalls de dados, flushes de controle, misses de cache e desequilíbrio entre estágios somam ciclos extras, então o ganho observado fica bem abaixo do teto teórico.



Após a sessão, como você avalia seu entendimento dos temas abaixo? *

Marque uma opção por linha.

Use a escala de 0 a 5, em que 0 = não entendi e 5 = entendo com segurança.

	0	1	2	3	4	5
Pipeline de 5 estágios	☐	☐	☐	☐	☐	☐
Hazards de dados e controle	☐	☐	☐	☐	☐	☐
Cache e hierarquia de memória	☐	☐	☐	☐	☐	☐
AMAT (Tempo Médio de Acesso à Memória), CPI (Ciclos Por Instrução) e speedup	☐	☐	☐	☐	☐	☐
Formatos de instrução e ISA (Arquitetura do Conjunto de Instruções) do RISC-V (arquitetura de conjunto de instruções aberta e reduzida)	☐	☐	☐	☐	☐	☐
Multi-core e estado compartilhado	☐	☐	☐	☐	☐	☐

Voltar

Enviar

Limpar formulário

Nunca envie senhas pelo Google Formulários.

Este conteúdo não foi criado nem aprovado pelo Google. - [Entre em contato com o proprietário do formulário](#) - [Termos de Serviço](#) - [Política de Privacidade](#)

Este formulário parece suspeito? [Denunciar](#)

Google Formulários



